

Entrega — DocViz / BSG (Sesión 1)

Asignatura: Diseño de infraestructura escalable
Alcance: Arquitectura del servicio en **AWS**, **uso de modelos de lenguaje y embeddings** en el pipeline RAG, cumplimiento de requisitos y referencias del código.

Acceso rápido (leer primero)

Qué	Enlace o dato
Aplicación DocViz (usuario final)	http://bsg-frontend-alb-2137705119.us-east-1.elb.amazonaws.com/
Vídeo demo (Google Drive)	DocViz — Sesión 1 (demo)
Versiones finales desplegadas (ECS)	Frontend 1.0.32 · DocViz backend 3.0.46 · back-security 1.0.9 (coinciden con version.txt y tags en Docker Hub / Terraform). Detalle en §11.2 .
Usuarios de prueba	Ver §11.3 (admin01 / soporte01).

1. Resumen ejecutivo

El sistema **DocViz** es una aplicación **API-first** formada por tres piezas desacopladas:

Pieza	Rol
Frontend	SPA React + Vite; Nginx sirve estáticos y proxifica /api y /security-api hacia los microservicios internos (mismo origen para el navegador).
back-security	Autenticación (JWT), usuarios y roles; WebFlux + R2DBC; integración con Redis y DynamoDB para revocación de tokens.
backend DocViz	API de dominio: Git, índice vectorial (pgvector en PostgreSQL), RAG (recuperación + generación con Spring AI), work area, soporte, S3, cabecera X-DocViz-User.

El núcleo del curso respecto a **IA aplicada** está en el backend DocViz: **embeddings** para indexar fragmentos del repositorio y **modelo de chat** para responder con contexto recuperado (véase **sección 4**).

La infraestructura en **AWS** separa entrada pública (ALB, API Gateway) de **ECS Fargate**, Cloud Map, RDS, Redis, DynamoDB y S3, definida en **Terraform** (terraform-project/main.tf). La elección de **AWS frente a Google Cloud** se argumenta en la **sección 5**.

2. Arquitectura lógica

 Diagrama de Arquitectura Lógica

Nota: Se adjunta diagrama exportado a imagen para correcta visualización en PDF.

Lectura rápida:

- El **usuario final** entra por el **ALB** → contenedor **frontend**: el JS llama a rutas relativas **/security-api** y **/api**; **Nginx** reescribe y envía a **security.bsg.internal:8081** y **docviz.bsg.internal:8080** (resolución DNS dinámica en VPC).
- API Gateway** expone URLs públicas del tipo `https://<api-id>.execute-api.<region>.amazonaws.com/security-auth/...` y `/docviz/...`, con **VPC Link** hacia los **NLB internos** que balancean a los mismos servicios ECS.

3. Componentes de software (repositorio)

Directorio	Tecnología	Responsabilidad
frontend-sesion1	React, TypeScript, Vite	UI; login → /security-api; API DocViz → /api; cabeceras X-DocViz-User y rol.
back-security-sesion1	Spring WebFlux, R2DBC	Login, registro, JWT; prefijo /security-auth tras el proxy.
backend-sesion1	Spring Boot 3, JDBC, Spring AI	Git, pgvector , RAG , S3, soporte; configuración de chat y embeddings por perfil Spring.

Docker / versionado: imágenes en Docker Hub; version.txt por módulo e imagen referenciada en Terraform (locals en main.tf).

4. IA generativa y RAG: embeddings, chat y perfiles Spring

Esta sección concentra lo pedido por el curso: **qué modelos usamos, para qué y cómo encajan en el flujo RAG**.

4.1 Idea general del pipeline RAG en DocViz

1. **Ingesta:** el texto del repositorio (y fuentes configuradas, p. ej. markdown de soporte en S3) se divide en fragmentos (**chunks**).
2. **Embeddings:** cada fragmento se vectoriza con el **modelo de embeddings** elegido y se guarda en **PostgreSQL con extensión pgvector** (búsqueda por similitud).
3. **Consulta:** la pregunta del usuario se embedding-a con el **mismo modelo familia/proveedor** que en indexación (coherencia dimensional y semántica).
4. **Recuperación:** se recuperan los **top-k** fragmentos más cercanos en el espacio vectorial.
5. **Generación:** esos fragmentos se inyectan como **contexto** en el prompt enviado al **modelo de chat**, que produce la respuesta final (con límites de historial y tamaño de contexto configurados en `application.properties`).

Todo el acoplamiento a proveedores externos está centralizado en **Spring AI** (`spring.ai.*`), lo que permite cambiar perfil sin reescribir la lógica de negocio del RAG.

4.2 Producción en AWS (perfil pdn)

En ECS el backend arranca con **SPRING_PROFILES_ACTIVE=pdn**. La configuración activa **OpenAI** para **chat y embeddings** vía API HTTPS (`api.openai.com`), sin Ollama en el contenedor (auto-config de Ollama excluida en `application-pdn.properties`).

Rol	Modelo	Detalle
Embeddings (índice + consulta RAG)	text-embedding-3-small	Modelo de embedding denso de OpenAI; dimensiones alineadas con docviz.vector.embedding-dimensions (p. ej. 768 en configuración actual para coincidir con el esquema pgvector).
Chat (respuesta al usuario con contexto recuperado)	gpt-4o-mini (por defecto)	Modelo conversacional vía Chat Completions; configurable con OPENAI_CHAT_MODEL . Límite de salida acotado con max-completion-tokens (p. ej. 8192) según API reciente.

Variables típicas: **OPENAI_API_KEY** (y equivalentes Spring AI para chat y embedding); el mismo secreto alimenta chat y embeddings en producción.

Por qué esta combinación para el curso: separa claramente **representación vectorial** (embeddings baratos y estables para miles de chunks) de **razonamiento lingüístico** (chat que sintetiza respuestas con citas de contexto). Es el patrón estándar en aplicaciones RAG empresariales sobre Spring AI.

4.3 Desarrollo local (perfil loca1)

Para no depender de API de pago en cada máquina, el perfil **loca1** usa **Ollama**:

Rol	Modelo por defecto
Chat	llama3.2 (override <code>OLLAMA_MODEL</code>)

Rol	Modelo por defecto
Embeddings	<code>nomic-embed-text</code> (768 dimensiones; debe coincidir con <code>docviz.vector.embedding-dimensions</code>)

La URL de Ollama es configurable (`OLLAMA_BASE_URL`, típicamente `http://127.0.0.1:11434`).

4.4 Perfil intermedio develop

Similar a producción en cuanto a **OpenAI**: `gpt-4o-mini` para chat y `text-embedding-3-small` para embeddings (`application-develop.properties`), útil para entornos de integración sin ECS.

4.5 Relación con la infraestructura

- El task **DocViz en ECS** tiene **más CPU y memoria** que el frontend porque **embeddings e inferencia** (y la ingesta de repositorios) concentran carga.
- Las **llaves de OpenAI** no se versionan: llegan por **Terraform / variables sensibles** al task definition o por secretos del entorno de CI/CD.
- **PostgreSQL + pgvector** es el almacén canónico del índice; la escalabilidad del **relleno del índice** y del **tráfico de chat** se apoya en escalar tasks ECS y dimensionar RDS según métricas.

5. Por qué AWS frente a Google Cloud Platform (GCP)

Elegimos **AWS** por criterios prácticos del curso, no porque sea intrínsecamente mejor que **GCP**: la arquitectura ya integraba ALB/NLB, ECS Fargate, API Gateway + VPC Link hacia NLB, RDS, ElastiCache, DynamoDB, S3 y Cloud Map con **Terraform** (`main.tf` en `us-east-1`), patrón documentado y alineado con el laboratorio docente; reproducir lo mismo en GCP supondría otro stack (p. ej. Cloud Run/GKE, balanceadores internos, Memorystore, Cloud SQL) y **otro ciclo de integración** sin sumar al objetivo pedagógico. Además el **chat y los embeddings** van por **OpenAI API** desde Spring, así que el modelo no viene de Bedrock ni de Vertex: la nube cubre sobre todo compute, red y datos ya montados en AWS. **GCP sería viable**, pero priorizamos centrarnos en **RAG, Spring AI y escalabilidad** en lugar de portar la topología a otro proveedor.

6. Infraestructura AWS (Terraform)

Recurso	Uso en el proyecto
ECS Fargate (<code>bsg-cluster</code>)	Tres servicios: <code>bsg-frontend-service</code> , <code>bsg-back-security-service</code> , <code>bsg-backend-service</code> ; logs en CloudWatch <code>/ecs/bsg-frontend</code> , <code>/ecs/bsg-back-security</code> , <code>/ecs/bsg-backend</code> .

Recurso	Uso en el proyecto
ALB (bsg-frontend-alb)	Entrada HTTP pública hacia el task del frontend (target group puerto 80).
NLB internos	TCP 8081 (security) y 8080 (DocViz); integración API Gateway por ARN de listener .
API Gateway HTTP API	Rutas /security-auth/{proxy+} y /docviz/{proxy+} → HTTP_PROXY + VPC Link .
Cloud Map (bsg.internal)	security.bsg.internal y docviz.bsg.internal .
RDS PostgreSQL	Bases bsg_security y docviz (esta última debe existir).
ElastiCache Redis	Security.
DynamoDB (bsg_revoked_tokens)	Tokens revocados; IAM en task role del security.
S3 (tres buckets)	Soporte, borradores, work area del backend DocViz.
IAM	Roles ECS y políticas DynamoDB / S3 según servicio.

Outputs útiles: frontend_alb_url, api_gateway_endpoint, rds_endpoint, redis_endpoint, DNS de NLB, buckets S3.

7. Cómo cumplimos los requisitos de infraestructura escalable

7.1 Separación de responsabilidades y microservicios

- **Seguridad** y **DocViz** son **despliegues distintos**, prefijos HTTP claros (/security-auth vs /docviz).
- El **frontend** solo sirve UI y proxy; la **IA** reside en el backend DocViz.

7.2 Escalabilidad horizontal y desacoplamiento

- **ECS Fargate** permite escalar réplicas por servicio (el backend DocViz puede escalarse según carga de **ingesta y chat**).
- **NLB, Cloud Map** y **API Gateway** desacoplan clientes de la ubicación exacta de los tasks.

7.3 Seguridad

- JWT en **back-security**; revocación en **DynamoDB**; secretos de OpenAI fuera del repo.

7.4 Persistencia y servicios gestionados

- **RDS + pgvector** para vectores y datos relacionales; **S3** para objetos; **Redis** para security.

7.5 Observabilidad

- **CloudWatch Logs** por servicio; health checks en balanceadores y ECS.

7.6 Infraestructura como código

- **Terraform** en terraform-project/main.tf.

7.7 Experiencia de usuario

- **Mismo origen** en el ALB para la SPA; CORS en API Gateway para llamadas directas al endpoint público.

8. URLs de referencia (producción AWS)

Tras terraform apply, usar outputs:

- **SPA:** frontend_alb_url.
- **API HTTP:** api_gateway_endpoint — prefijos /security-auth/... y /docviz/....

La URL pública y el vídeo se repiten en detalle en la **sección 11**; la referencia principal está arriba en **Acceso rápido**.

9. Despliegue de aplicación

- Build/push de imágenes según **DOCKERHUB.md**.
- Actualizar **task definitions** y servicios ECS; frontend con DOCVIZ_UPSTREAM, SECURITY_UPSTREAM, NGINX_RESOLVER acordes a Cloud Map.

10. Documentación relacionada

Documento	Contenido
README.md	Monorepo y docker compose.
DOCKERHUB.md	Imágenes y build/push.
terraformom-project/main.tf	Recursos AWS.

Documento	Contenido
backend-sesion1/src/main/resources/application-pdn.properties	OpenAI en producción.
backend-sesion1/src/main/resources/application-local.properties	Ollama en local.
README por proyecto	Detalle por componente.

11. Anexo para el evaluador: URL pública, vídeo en Drive y credenciales de prueba

(Resumen inicial en Acceso rápido.)

11.1 URL de la aplicación (usuario final)

DNS del ALB del frontend en AWS (frontend_alb_url).

Campo	Valor
URL pública DocViz (SPA)	http://bsg-frontend-alb-2137705119.us-east-1.elb.amazonaws.com/

11.2 Vídeo y versiones finales de los servicios

Vídeo (Google Drive)

Demostración de la plataforma (acceso público, login, flujos por rol, repositorio / RAG / chat). Enlace con permiso de visualización para evaluación:

Campo	Valor
Enlace al vídeo	Ver en Google Drive

Versiones finales desplegadas (imágenes ECS / Docker Hub)

Referencias en Terraform (terraform-project/main.tf, locals) y en cada version.txt del monorepo:

Servicio	Tag / versión	Archivo version.txt
Frontend (Nginx + SPA)	1.0.32	frontend-sesion1/version.txt
DocViz backend	3.0.46	backend-sesion1/version.txt

Servicio	Tag / versión	Archivo version.txt
back-security	1.0.9	back-security-sesion1/version.txt

Imágenes Docker Hub usadas por ECS: mmercado94/frontend-sesion1:<tag>, mmercado94/backend-sesion1:<tag>, mmercado94/back-security-sesion1:<tag> (mismo número que la tabla).

11.3 Credenciales para pruebas (admin01 y soporte01)

Estos usuarios están definidos en el **seed** de desarrollo del microservicio de seguridad (back-security-sesion1, data.sql). **Antes de la demo**, comprobad que la base **RDS** del entorno desplegado los contiene (misma semilla o migraciones aplicadas).

Usuario	Contraseña	Rol
admin01	Admin123	Administrador (ROLE_ADMINISTRATOR) — flujo típico admin/cells
soporte01	Soporte123	Soporte (ROLE_SUPPORT) — flujo típico support/cells

Nota: Son credenciales pensadas para **demonstración académica**. En un despliegue real conviene **rotar contraseñas**, usar usuarios provisionados de forma controlada y **no** versionar secretos en repositorios públicos.

Documento para entrega académica — alineado al repositorio, Spring AI y Terraform.